## Pulse sequence generation

The MIPS system has the capability to generate intricate pulse sequences. Users can program pulse sequences by defining digital outputs and DC bias channel outputs, which are set at predetermined times.

Pulse sequences (also referred to as tables) are defined by a series of events, each occurring at a specific time and corresponding to a set of output values that are set at the event time. The user specifies a set of events to construct a complete pulse sequence. The user-defined events can then be placed in a loop to cause the sequence to repeat for a user-defined number of cycles.

An event is defined by an ASCII character string as shown below:

Count:Channel:Value

The Count defines the time in clock cycles when the event occurs. The clock can be programmed to several different internal clock sources or an external clock can be used. The Channel defines the MIPS output to set at the defined Count. The Value parameter is assigned to the Channel at the events Count value.

Multiple Channel:Value pairs can be defined at a single Count event time point. Below are several examples of valid events.

| | |
|---|---|
| 10:A:1 | Set digital output A to logic 1 at 10 clock cycles after the table starts. |
| 25:A:0:5:34.5 | Set digital output A to logic 0 and DC bias channel 5 to 34.5 volts at 25 clock cycles after the table starts. |

A pulse sequence can be created by defining multiple events that are comma separated as shown in the example below.

10:A:1,25:A:0:5:34.5

In this example the two events described earlier are used to form a pulse sequence that consists of two separate events.

A set of events can be repeated in a loop by enclosing them in square brackets. The start and end of the loop are defined by these brackets. The syntax for a loop is illustrated below:

[name:cycles,event1,event2,length:]

The name parameter, a single character, specifies the name of the sequence within the loop. The cycles parameter determines the number of iterations the loop will execute. Setting cycles to zero causes the loop to repeat indefinitely. The total duration of the loop in clock

cycles is determined by the length parameter. Below is an example of a loop using the two events defined above. The loop will repeat 10 times and has a period of 100 clock cycles.

[A:10,10:A:1,25:A:0:5:34.5,100:]

To execute the loop, you must define the starting point in clock cycles after the pulse sequence generator has commenced. Below, you will find a fully formed pulse sequence that commences 25 clock cycles after the pulse sequence generator is triggered.

25:[A:10,10:A:1,25:A:0:5:34.5,100:]

Multiple looping sequences can be arranged in a series, and loops can be nested up to a maximum depth of five levels.

The table below enumerates all the permissible channel parameters that can be utilized within a pulse sequence event and describes the valid values parameter range.

**Table 1, Channel value options**

| ASCII value | Decimal value | Description |
|---|---|---|
| A thru P | 65 thru 80 | MIPS digital outputs. These channels can have values of 0 or 1. |
| | 1 thru 32 | MIPS DC bias output channels. These channels can have values that are within the valid range of the DC bias module installed in the MIPS system |
| | 33 thru 36 | MIPS RF driver channels. These channels define RF driver drive levels for RF drivers channel 1 through 4. |
| t | 116 | This defines the MIPS trigger output. The value parameter defines the output pulse width in µs. If the value is 0 then the output is set to logic low if the value is -1 then the output is set to logic high. |
| d | 100 | This channel specifies a time delta increment value. The value determines the increment value in clock cycles and can be positive or negative. The subsequent time parameters that are defined as negative will be adjusted by this value. |
| p | 112 | This channel defines a time delta offset value. The value specifies the offset value in clock cycles and can be positive or negative. The subsequent time parameters that are defined as negative will be adjusted by this value. |
| r | 114 | This channel will set the ARB compression line, valid values are 0 or 1. This requires an ARB module in your MIPS system. |
| s | 115 | This channel will set the ARB sync line, valid values are 0 or 1. This requires an ARB module in your MIPS system. |
| b | 98 | This channel will trigger the MIPS burst generator to execute a pre-programmed burst sequence. The value parameter is ignored. |
| c | 99 | This channel will trigger the ARB compression table, the value parameter is ignored. A MIPS ARB module is required to use this channel. |
| e thru h | 101 thru 104 | These channels set the ARB module Aux output values for ARB modules 1 through 4. The valid value range is +/-50 and require the ARB module in the MIPS system. |
| i thru l | 105 thru 108 | These channels set the ARB module offset values for ARB modules 1 through 4. The valid value range is +/-50 and require the ARB module in the MIPS system. |

## Dynamic time point adjustments

The pulse sequence table supports dynamic time points, enabling a pulse sequence to modify the value of a time point defined in clock cycles. If a time point is negative, it is flagged for adjustment by the ‘d’ and ‘p’ channel commands. The absolute value of the time point determines its actual value, while the negative value indicates that this value will be adjusted after the ‘d’ and/or ‘p’ channel command is processed.

This capability enables the definition of a pulse sequence loop that sequentially sweeps a pulse in time as the loop progresses. An illustrative example of a similar pulse sequence loop, as defined previously, is provided below. In this example, the pulse is shifted by one clock cycle for each iteration through the sequence.

0:[A:10,-10:A:1,-25:A:0:5:34.5,100:d:1:]

To ensure proper synchronization, the user must verify that the two pulse sequence events do not collide or overlap. Additionally, all clock cycle time values within a pulse sequence must be arranged in ascending order.

## MIPS host application timing generator

Pulse sequences can be generated manually or programmatically using the scripting system in MIPS. The MIPS timing generator control allows for the generation of pulse sequences that include looping (1 level) and dynamic time point adjustments. It also enables the selection of trigger modes, clocking options, and the definition of time points in milliseconds. This is an innovative way to generate pulse sequences that will meet most needs. The following sections describe advanced capabilities that are only available when generating your own pulse sequence.

## Conditional logic loop

A pulse sequence loop designated as ‘P’ is enabled for conditional logic control. In this mode, a channel:value pair in an event can be preceded by conditional logic that utilizes the loop counter value. In this mode, the event definition can contain conditional logic. Below are a few examples of events with conditional logic:

| | |
|---|---|
| =:25:4:12.5 | In this example if the loop counter is equal to 25 then MIPS DC bias channel 4 is set to 12.5 volts. |
| >:25:5:23.7 | In this example if the loop counter is greater than 25 then MIPS DC bias channel 5 is set to 23.7 volts. |
| <:17:6:67.9 | In this example if the loop counter is less than 17 then MIPS DC bias channel 5 is set to 67.9 volts. |
| >:25:<:30:7:43.2 | In this example if the loop counter is greater than 25 and less than 30 then MIPS DC bias channel 7 is set to 43.2 volts. |

The conditional logic loop defined with the ‘P’ table name is exclusively applicable to DC bias channels. The digital output channels and other channel options enumerated in the channel table are not permitted for use. Three distinct special cases are defined below and can be utilized.

| | |
|---|---|
| Channel = 40 | Activates the ADC system. The ADC system must be preconfigured. This trigger signal initiates the acquisition function of the ADC, resulting in a vector of data recorded and saved by the ADC system. The value parameter is not utilized. |
| Channel = 41 | Sets the trigger output to logic level 1. The value parameter is not utilized. |
| Channel = 42 | Sets the trigger output to logic level 0. The value parameter is not utilized. |

Conditional logic applies exclusively to a single channel:value pair that is defined after the logic test. Any additional channel:value pairs will be applied without the logic test. The following example illustrates this concept:

| | |
|---|---|
| =:25:4:12.5:5:6.5 | In this example the logic test is applied to the MIPS DC bias channel 4 while channel 5 is set to 6.5 regardless of the logic test. |
| =:25:4:12.5: =:25:5:6.5 | In this example the logic test is repeated for MIPS DC bias channel 5. |

Voltage ramping can be implemented using conditional logic. DC bias channels that have the ramp flag bit set in their channel parameter will be adjusted by the channel’s value parameter. This enables conditional loops to generate voltage ramps, with a maximum of eight channels utilizing this feature within a single conditional loop. To utilize this feature, the channels intended for ramping must be initialized for ramping, and the pulse sequence generator must have this ramping feature enabled. Please refer to the example provided below for further clarification.

The MIPS command, STBLVDLT,TRUE will enable the ramping mode and only needs to be issued one time.

Two flags are applied to the channel values to enable ramping. These values are added to the channel number. For instance, if channel 1 is flagged to ramp, it becomes 129; if flagged for initialization, it becomes 65.

| | |
|---|---|
| RAMP | = 0x80 or 128 |
| INITIAL | = 0x40 or 64 |

The example pulse sequence shown below will initialize channel 1 for ramping and set its initial value. This value will be increased by 1 volt each time through the conditional loop.

0:65:0:[P:10,0:129:1,100:]

In this example, 65:0 initializes channel 1 for ramping and sets its initial value to 0. In the conditional loop, 129:1 increases the voltage on channel 1 by 1 volt each time through the loop. The loop will repeat 10 times, defined by P:10, resulting in a voltage ramp from 0 to 10 volts generated on channel 1. The rate of the ramp is defined by the loop length, in this case, 100. This is the number of clock cycles that define the loop length. The ramp voltage adjustment can include a conditional logic test as well. In the example below, the added conditional logic will cause the ramp to start after 5 cycles and ramp from 0 to 4 volts.

0:65:0:[P:10,0:>5:129:1,100:]

The ramp generation can be repeated by adding an additional loop that defines the number of times the ramp is generated. The example below adds this additional loop. The ramp is generated 10 times, defined by A:10 where A is the table name. The period of the repeat is defined by table A's length parameter, in this case 10,000.

0:[A:100,0:65:0:[P:10,0:>5:129:1,100:],10000:]

These features enable the generation of multiple ramps with varying ramp rates in a pulse sequence. A single channel can encompass multiple ramping sections, each with distinct ramp rates. This capability allows the definition and generation of intricate sequences by the pulse sequence generation system.

## Ramp generation

The pulse sequence generation system can generate voltage ramps without employing the conditional logic loop (table-based ramping). This process utilizes a distinct ramping mechanism and necessitates defining the ramp adjustment frequency through commands issued to MIPS, as well as activating the ramping system. The following command issues to MIPS will enable this ramping capability and establish the ramp adjustment rate at 2 kHz.

STBLRMPENA,TRUE,2000

A maximum of three ramps can be defined within a single sequence using this capability. The ramping functions are activated by setting specific flag bits in the channel numbers. These flag bits are:

RAMP = 0x80 or 128
INITIAL = 0x40 or 64

There are only two valid sets of flag options in this mode:

RAMP (0x80 or 128) and RAMP | INITIAL (0xC0 or 192).

The RAMP | INITIAL flag initializes a channel for ramping, and the value specifies the initial ramp starting voltage. This initialization flag must be present in the sequence to enable the ramp and can only occur once in the sequence.

The RAMP flag instructs the system to utilize the voltage value as the ramp delta. This value can be modified at any point within the sequence. Setting the value to zero will terminate the ramping process.

The RAMP | INITIAL and RAMP flags can coexist at the same time point, provided that the RAMP | INITIAL flag appears first. The following example demonstrates this:

193:0:129:1

In this case channel 1 is initialized to 0 volts and ramping is started at 1 volt per clock period. If the frequency was set to 1000 Hz then a ramp is 1000 volts per second would be generated.

To prevent unpredictable output, the table must be valid. The system does not test the voltages; it is the code generating the tables' responsibility to ensure their validity.

The provided example illustrates the generation of a ramp on the DC bias channel 2, comprising two distinct ramping sections. Initially, at time point 0, channel 2 is configured for ramping, with an initial voltage set to 1 volt. The ramp step voltage is subsequently set to 4 volts per update. At time point 5000, the ramp on DC bias channel 2 undergoes a change, transitioning to -1 volt per update, thereby reversing its direction. Subsequently, at time point 10000, the ramp is reinitialized, and its voltage is set to 0, effectively stopping the ramp. This ramp is generated in a loop table named 'A' with a repeat count of 4 and a period of 20000 clock cycles.

0:[A:4,0:194:1:130:4,5000:130:-1.0,10000:130:0,20000:];

## Limitations

MIPS generates pulse sequences utilizing an internal timer counter peripheral equipped with a compare register. When the compare register's value aligns with the timer's count value, an output pulse is generated. This hardware-based process does not rely on software to produce the output pulse. Subsequently, this pulse serves as a latch to apply DAC values and digital output values previously loaded into the registers.

Upon the generation of the output latch signal, the processor initiates an interrupt processing routine and loads all registers prior to the subsequent latch pulse. The processor's update speed serves as the primary limiting factor in this pulse generation scheme. The intricacy of the sequence and the number of values undergoing changes between events can lead to an overrun of the update process.
For reference, a general guideline for developing pulse sequences is provided below:

- The minimum time interval between two events cannot be less than 10 microseconds. Determine the number of values that are changing between timepoints and multiply this value by 20 microseconds to establish a safe starting point for the minimum time interval between the events.
- Ensure that the time points are in ascending order.
- Begin debugging by setting the time point deltas conservatively and gradually decreasing them after thoroughly debugging your sequence.
- Whenever feasible, utilize the MIPS control panel pulse sequence generation tool, which will significantly simplify this process and be effective for most applications.

## Running pulse sequences

Using the MIPS pulse sequence generator requires developing the pulse sequence as described in this document, as well as configuring the system to accept and execute the pulse sequence. The MIPS system has two operating modes: local (LOC) and table (TBL). These modes are set using the SMOD command.

Local mode

In local mode MIPS is monitoring all parameters and responding to commands from the host computer. The front panel user interface is active and the user can interact with MIPS through the front panel. In this mode the system will accept command to configure the pulse sequence generator and accept new pulse sequences.

Table mode

In table mode MIPS is running a table (pulse sequence) or waiting for a trigger to start a table that has been loaded. The MIPS front panel is disabled and a message box is displayed indicating the system is in table mode. The table mode can be aborted by pressing the MIPS front panel control knob.

The steps required to load and execute a pulse sequence are outlined below.

- Start with MIPS in LOC mode
- Generate a pulse sequence and download with the STBLDAT command, this command is semicolon terminated and must be entered within 3 seconds. If you are doing this process manually copy the pulse sequence from a text editor and paste it into the MIPS terminal interface
- Set the clock options using the STBLCLK command
- Set the trigger option using the STBLTRG command
- This represents the minimum required configuration and you can now enter table mode with the command SMOD,TBL
- MIPS should respond with a TBLRDY message, send the command TBLSTRT to run the pulse sequence

Pulse sequences that intend to generate voltage ramps need to use one of the two ramp enable commands depending on the ramping scheme used.

Multiple pulse sequences can be downloaded to MIPS and commands allow the user to select the pulse sequence to be used as well as automatically advancing when a pulse sequence completes.

Table 2, Command notations

| Notation | Meaning |
|---|---|
| [ ] | **Optional:** The parameter is not required for the get command to execute. |
| < > | **Variable:** Replace the brackets and text with a specific value. |
| \| | **Pipe**: Indicates a **logical OR**. You must choose one of the options separated by the bar. |

Table 3, Pulse Sequence Generator Commands

| Command | Parameters | Description |
|---|---|---|
| **STBLDAT** | <pulseSequenceString> | Load the pulse sequence string. The system must be in local mode to load a pulse sequence |
| **STBLCLK** | <EXT\|EXTN\EXTS\| 42000000\|10500000\| 2625000\|656250> | Clock source, EXT=Q clock input positive edge, EXTN=Q clock input negative edge, EXTS= S input positive edge, numeric values are internal clock rates. |
| **STBLTRG** | <SW\|POS\|NEG\|EDGE> | Trigger source, SW=software trigger, POS=R input positive edge, NEG=R input negative edge, Edge=R input any edge. |
| **TBLSTRT** | | Software trigger, starts a ready pulse sequence. |
| **TBLSTOP** | | Stop table, gracefully stops a running pulse sequence. |
| **TBLABRT** | | Abort table, forcefully stops a running pulse sequence. |
| **SMOD** | <LOC\|TBL\|ONCE> | Pulse sequence mode. LOC = local mode, no pulse sequence activity. TBL = table mode, pulse sequence ready to run. ONCE = Table mode active, when table execution finished exit to LOC mode. |
| **GTBLFRQ** | | Current clock frequency |
| **STBLNUM/GTBLNUM** | [<number>] | Active table number, range 1 thru 5 |
| **STBLADV/GTBLADV** | [<TRUE\|FALSE>] | Auto-advance table number |
| **STBLVLT** | <time point>,<channel>, <value> | Modify voltage in loaded table |
| **GTBLVLT** | <time point>,<channel> | Read table entry voltage |
| **STBLCNT** | <time point>,<channel>, <new time point> | Modify time point value |
| **STBLDLY** | <ms> | Inter-table delay |
| **STBLREPLY/GTBLREPLY** | [<TRUE\|FALSE>] | Table response messages |
| **SOFTLDAC** | <TRUE\|FALSE> | Forces system to generate the latch signal in the ISR |

| | | |
|---|---|---|
| **STBLTSKS/GTBLTSKS** | [<TRUE\|FALSE>] | Performs an update of system tasks in idle areas of a pulse sequence |
| **SEXTFREQ/GEXTFREQ** | [<frequency>] | When using an external clock this command defines the external clock frequency |
| **STBLUSBTST/ GTBLUSBTST** | [<TRUE\|FALSE>] | Test the USB link in the pulse sequence table process loop |
| **STBLVDLT/GTBLVDLT** | [<TRUE\|FALSE>] | Enable the ramp generation when using the conditional loops |
| **TBLCHK** | | Performs a table timing check |
| **STBLRMPENA** | <TRUE\|FALSE>,<freq> | Enables table-based ramping and sets the update frequency for all ramps |
| **STBLDLT/GTBLDLT** | [<delta>] | Delta value access that is used to adjust negative time points |
| **GTBLSTA** | | Report table status:<br>IDLE<br>READY<br>TRIGGERED<br>ABORTED |
| **STBLEVY/GTBLEVY** | [<TRUE\|FALSE>] | Enables trigger after each table segment |
| **TBLTSKENA** | <TRUE\|FALSE> | Enable all tasks update, only use if all boards in system are using address A |
| **STBLRETRIG/ GTBLRETRIG** | [<TRUE\|FALSE>] | Enables table external trigger re-triggering |